Comparable vs Comparator

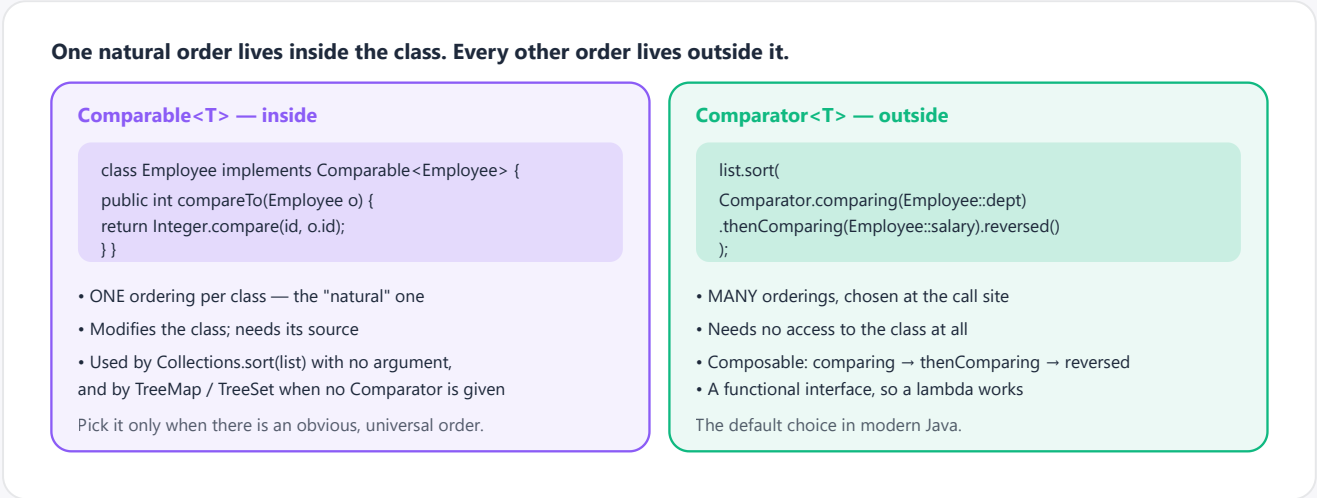
9 min read Updated 28 Jul 2026 Topic 25 of 65

Why it matters

Sorting comes up in almost every interview, usually as "sort these employees by department, then salary descending". The answer separates people who write a fifteen-line anonymous class from people who write one composed line — and the composed version is not just shorter, it is harder to get wrong. This topic also closes a loop from Chapter 4: `TreeMap` and `TreeSet` decide equality with `compareTo`, so a broken comparator silently loses data.

The concept, simply

Both define an ordering; the difference is *where the ordering lives*.



Comparable is a property of the class — one natural order, baked in. Comparator is a separate object, so a class can be sorted many different ways without being modified.

- `Comparable<T>` — implemented *by* the class, defining its single *natural order*. One method: `int compareTo(T other)`.

- `Comparator<T>` — a separate object defining *an* order. One abstract method: `int compare(T a, T b)`. You can have as many as you like, and you need no access to the class.

Both return a negative number, zero, or a positive number for "less than", "equal", "greater than". Nothing requires -1 and 1 specifically.

How it works

	`Comparable`	`Comparator`
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T a, T`
Where	Inside the class	Outside, at the ca
How many	One per class	Unlimited
Needs the source?	Yes	No
Lambda-able	No	Yes — a functiona
Used by	`sort()` with no argument, `TreeMap` / `TreeSet` by default	`sort(cmp)`, `new`

The contract is what interviewers probe, because breaking it causes bugs that are hard to trace. An ordering must be:

- **Antisymmetric** — `sgn(compare(a,b)) == -sgn(compare(b,a))`.
- **Transitive** — if a > b and b > c then a > c.
- **Consistent** — equal elements compare equally against everything else.

4's `TreeMap` merge).

Break the contract and `Collections.sort` may throw `IllegalArgumentException: Comparison method violates its general contract!` — a real error from the TimSort implementation detecting inconsistency mid-sort. It usually appears only on large inputs, which is why it tends to surface in production rather than in tests.

Code examples

Modern `Comparator` composition, which is the answer to "sort by X then Y":

JAVA Copy

```java
record Employee(String name, String dept, int salary) { }

List<Employee> staff = new ArrayList<>(…);

// Department ascending, then salary DESCENDING within each department.
staff.sort(Comparator.comparing(Employee::dept)
                     .thenComparing(Employee::salary, Comparator.reverseOr

// Careful — .reversed() at the END reverses the WHOLE chain, not the last
staff.sort(Comparator.comparing(Employee::dept)
                     .thenComparing(Employee::salary)
                     .reversed());     // dept DESC and salary DESC

// Primitive-specialised versions avoid boxing on every comparison:
staff.sort(Comparator.comparingInt(Employee::salary));

// Null-safe, which hand-written comparators almost always forget:
staff.sort(Comparator.comparing(Employee::name,
        Comparator.nullsFirst(Comparator.naturalOrder())));
```

`Comparable` for a genuine natural order:

JAVA Copy

```
    @Override
    public int compareTo(Version o) {
        // ✓ Integer.compare — no overflow, and it reads clearly.
        int c = Integer.compare(major, o.major);
        if (c ≠ 0) return c;
        c = Integer.compare(minor, o.minor);
        if (c ≠ 0) return c;
        return Integer.compare(patch, o.patch);
    }
}
// Now sorting needs no argument, and TreeSet<Version> just works:
Collections.sort(versions);
Set<Version> sorted = new TreeSet<>(versions);
```

The subtraction bug — the single most common comparator mistake:

JAVA Copy

```
// ✗ Overflows. Looks fine, passes every small test, fails on real data.
Comparator<Employee> broken = (a, b) → a.salary() - b.salary();

int a = Integer.MAX_VALUE, b = -1;
System.out.println(a - b);                    // -2147483648  → says a < b
System.out.println(Integer.compare(a, b));    // 1            → correct ✓

// ✓ Always Integer.compare / Long.compare / Double.compare.
Comparator<Employee> fixed = Comparator.comparingInt(Employee::salary);

// Double has a second reason: Double.compare handles NaN and ±0.0 correct
// which naive subtraction and even < / > do not.
```

And the inconsistency that throws mid-sort:

JAVA Copy

```
// ✗ Not antisymmetric — compare(a,a) returns 1, and TimSort will detect i
Comparator<Task> bad = (x, y) → x.priority() ≥ y.priority() ? 1 : -1;
```

```
// java.lang.IllegalArgumentException: Comparison method violates its gene
// ✓ Three-way, so equal really means 0.
Comparator<Task> good = Comparator.comparingInt(Task::priority);
```

Under the hood

What actually sorts. `Collections.sort` and `List.sort` delegate to `Arrays.sort`, which for objects uses **TimSort** — a stable, adaptive merge sort that exploits runs already in order, giving O(n) on nearly-sorted input and O(n log n) worst case. *Stable* means equal elements keep their relative order, which is exactly what makes "sort by salary, then re-sort by department" work as a two-pass approach. For primitives, `Arrays.sort` uses a dual-pivot quicksort instead — faster, but not stable, which does not matter because primitives are indistinguishable when equal.

Why TimSort detects contract violations rather than producing a wrong answer: it merges runs assuming a consistent ordering, and an inconsistent comparator can drive its internal invariants into an impossible state. Rather than corrupt the array silently, it throws. The check only fires on inputs large enough to trigger merging — typically 32 or more elements — which is precisely why these bugs skip past unit tests.

`Comparator` **is a functional interface** with one abstract method, which is why a lambda works and why the whole `comparing` / `thenComparing` / `reversed` family exists as default and static methods on it. That family is the reason to prefer `Comparator` for anything beyond a single obvious ordering: it composes, it is null-safe on request, and it reads like the requirement it implements.

Common mistakes

- **Subtraction in a comparator.** `a - b` overflows. Use `Integer.compare`.
- **A comparator that never returns 0.** Breaks antisymmetry and throws mid-sort on large inputs.
- `.reversed()` **at the end of a chain.** It reverses everything before it. Pass `Comparator.reverseOrder()` to the specific key instead.

- **Ignoring nulls.** A null field throws `NullPointerException` inside the sort. Use `nullsFirst` / `nullsLast`.
- **Ordering inconsistent with `equals`.** A `TreeSet` will silently treat unequal elements as duplicates (Chapter 4).
- **Implementing `Comparable` when there is no natural order.** If "which order?" has more than one sensible answer, it belongs in a `Comparator`.
- **Sorting an immutable list.** `List.of( ... )` and `Arrays.asList` results throw on `sort`. Copy first.
- **Boxing in hot comparisons.** Use `comparingInt` / `comparingLong` / `comparingDouble`.

Interview lens

Q: "`Comparable` vs `Comparator`?"
A: `Comparable` is implemented by the class itself and defines its one natural ordering via `compareTo` — it is what `Collections.sort(list)` and `TreeSet` use by default. `Comparator` is a separate object defining any ordering via `compare(a, b)`; you can have many, you do not need the class's source, and because it is a functional interface you can write it as a lambda and compose it.

Q: "Sort employees by department, then by salary descending."
A: `list.sort(Comparator.comparing(Employee::dept).thenComparing(Employee::salary, Comparator.reverseOrder()))`. The important detail is passing `reverseOrder()` to that specific key — putting `.reversed()` at the end of the chain would reverse the department ordering too.

Q: "What is wrong with `(a, b) -> a.getAge() - b.getAge()`?"
A: Integer overflow. If the values straddle the int range the subtraction wraps and the sign flips, so the comparator reports the opposite order. `Integer.compare(a.getAge(), b.getAge())` is correct and just as readable. With

doubles there is a second reason to use `Double.compare` — it handles NaN and ±0.0 properly.

Q: "What causes 'Comparison method violates its general contract'?"

A: An inconsistent comparator — usually one that never returns 0, or is not antisymmetric or transitive. TimSort merges assuming a consistent ordering, and when the invariants become impossible it throws rather than silently corrupting the array. It only fires on inputs large enough to trigger merging, which is why it typically escapes unit tests and appears in production.

Q: "What algorithm does Java use to sort?"

A: TimSort for objects — a stable, adaptive merge sort, O(n log n) worst case and close to O(n) on nearly-sorted data. Dual-pivot quicksort for primitive arrays, which is faster but unstable; that is fine because equal primitives are indistinguishable. Stability is why you can sort by one key and then another and keep the first as a tie-break.

Q: "Should a comparator be consistent with `equals`?"

A: It is strongly recommended, not required. Sorted collections use only the comparison, so an ordering that returns 0 for objects `equals` considers different will make a `TreeSet` drop them as duplicates and a `TreeMap` overwrite entries. That is sometimes deliberate — a case-insensitive map — but it must be a decision, not an accident.

Cheat-card

- `Comparable` = inside the class, ONE natural order, `compareTo(o)`, `java.lang`.
- `Comparator` = outside, MANY orders, `compare(a,b)`, `java.util`, lambda-able.
- Compose: `comparing().thenComparing().reversed()` — `reversed()` **flips the whole chain**.

- Violations throw *"Comparison method violates its general contract"* — only on big inputs.
- Objects sort with **stable TimSort**; primitives with dual-pivot quicksort (unstable).
- Nulls → `nullsFirst` / `nullsLast`. Hot paths → `comparingInt` / `Long` / `Double`.

QUICK CHECK — no marks, just for you.

1. What is wrong with the comparator (a, b) -> a.getAge() - b.getAge()?

- ◯ It returns the wrong sign for equal values
- ◯ Integer subtraction can overflow when the values straddle the int range, flipping the sign and reversing the order
- ◯ It fails to compile without an explicit cast
- ◯ Nothing — it is the idiomatic form

Check answer

2. comparing(Employee::dept).thenComparing(Employee::salary).reversed() sorts by…

- ◯ Department ascending, salary ascending
- ◯ Department ascending, salary descending
- ◯ Department DESCENDING and salary DESCENDING — reversed() flips the entire chain, not just the last key
- ◯ Salary descending only

Check answer

- Comparing objects of different classes
- Passing null to sort()
- Sorting an immutable list
- An inconsistent comparator — commonly one that never returns 0, or is not antisymmetric or transitive — detected by TimSort while merging

Check answer

‹ Previous
Bounded Types and Wildcards (PECS)

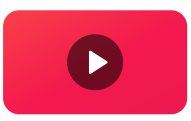
Next ›
The Throwable Hierarchy

LEARN MORE

ARTICLES & DOCS

- Comparator — API docs
  docs.oracle.com
- Comparable — API docs
  docs.oracle.com
- Object ordering — The Java Tutorials
  docs.oracle.com

VIDEOS



"comparable vs comparator" — video explainers
YouTube



comparable vs comparator interview questions

Java Interview Kit